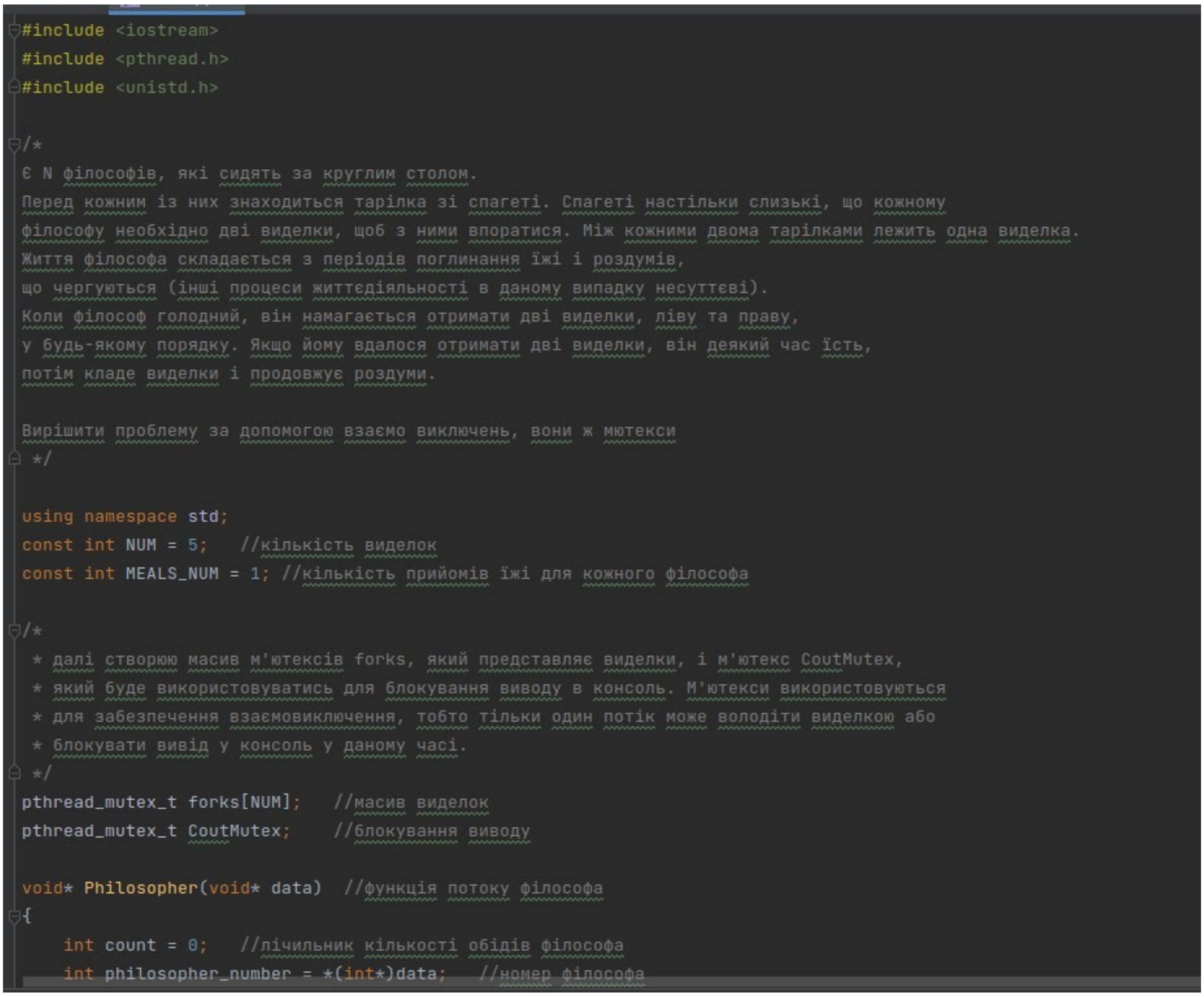
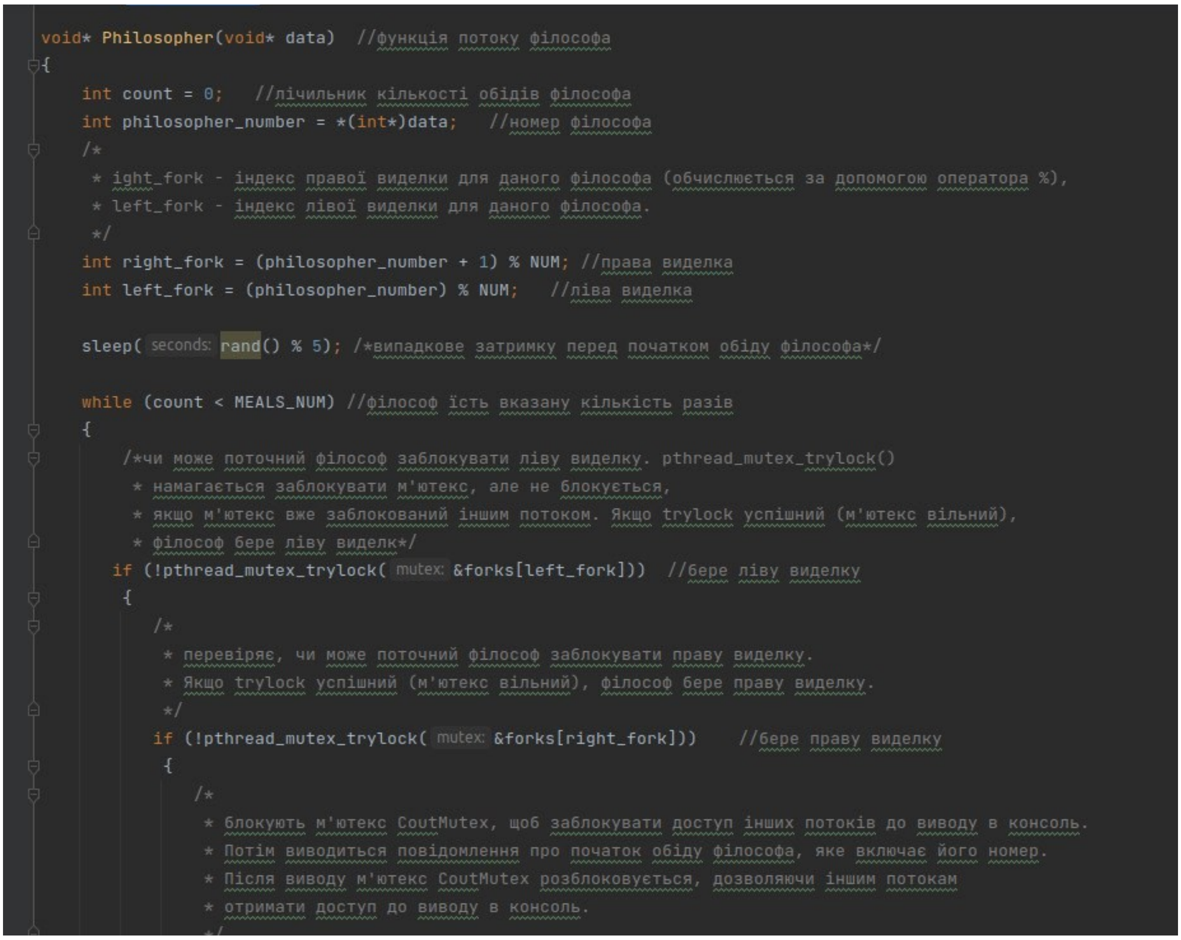
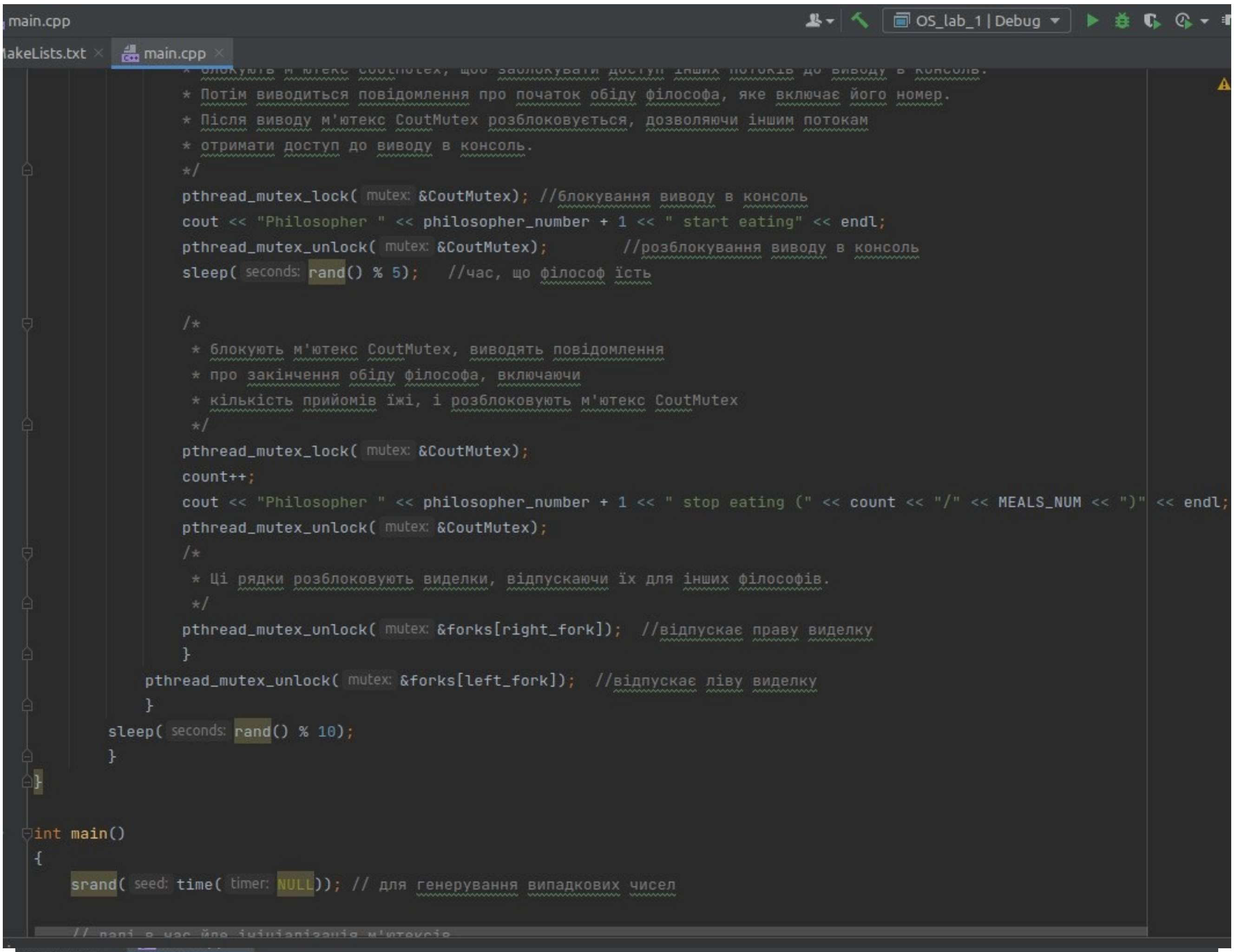
1.2



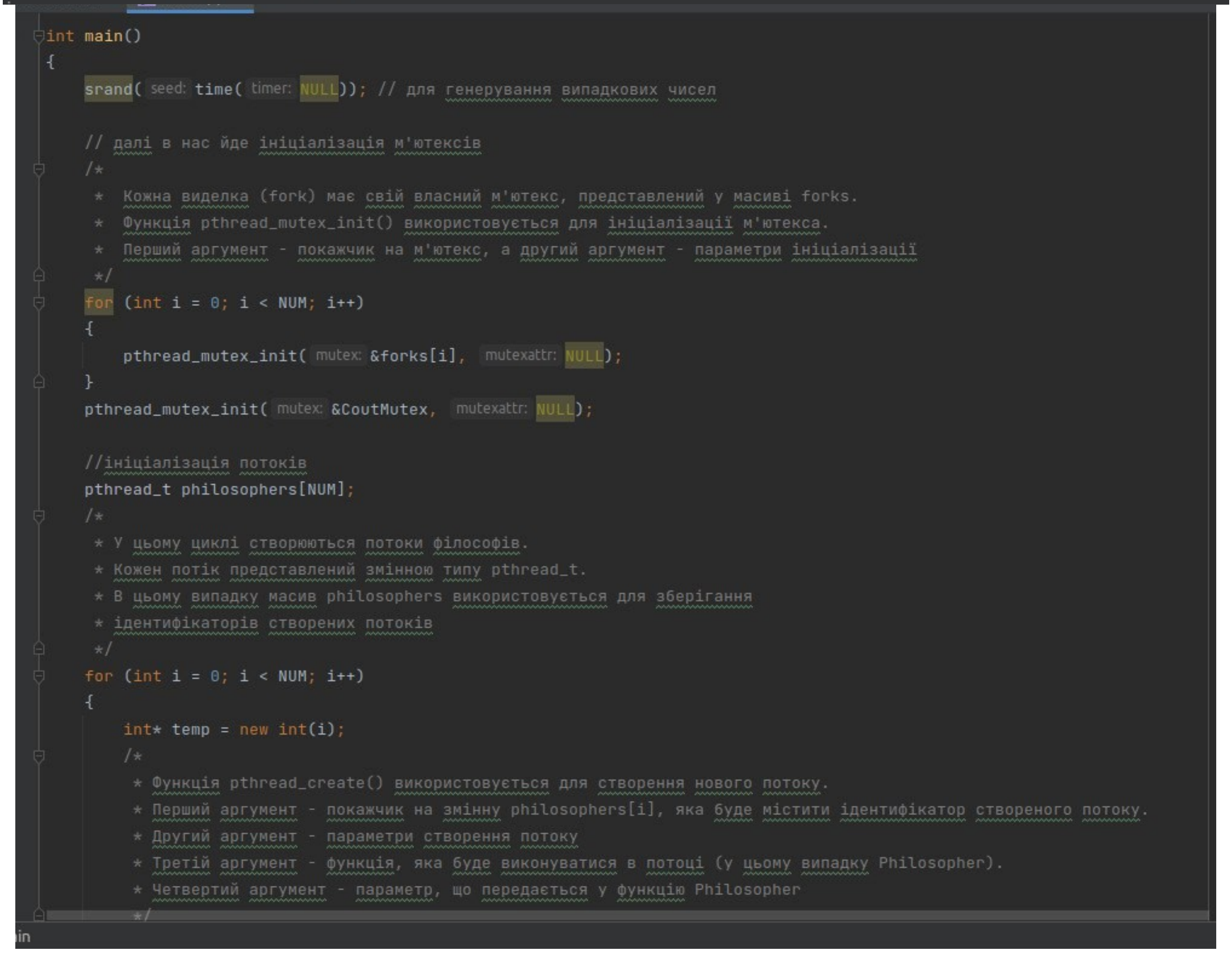
```
#include <iostream>
#include <pthread.h>
#include <unistd.h>

/*
Є N філософів, які сидять за круглим столом.
Перед кожним із них знаходиться тарілка зі спагеті. Спагеті настільки слизькі, що кожному
філософу необхідно дві виделки, щоб з ними впоратися. Між кожними двома тарілками лежить одна виделка.
Життя філософа складається з періодів поглинання їжі і роздумів,
що чергуються (інші процеси життєдіяльності в даному випадку несуттєві).
Коли філософ голодний, він намагається отримати дві виделки, ліву та праву,
у будь-якому порядку. Якщо йому вдалося отримати дві виделки, він деякий час їсть,
потім кладе виделки і продовжує роздуми.

Вирішити проблему за допомогою взаємо виключень, вони ж мютекси
 */

using namespace std;
const int NUM = 5;   //кількість виделок
const int MEALS_NUM = 1; //кількість прийомів їжі для кожного філософа

/*
 * далі створюю масив м'ютексів forks, який представляє виделки, і м'ютекс CoutMutex,
 * який буде використовуватись для блокування виводу в консоль. М'ютекси використовуються
 * для забезпечення взаємовиключення, тобто тільки один потік може володіти виделкою або
 * блокувати вивід у консоль у даному часі.
 */
pthread_mutex_t forks[NUM];   //масив виделок
pthread_mutex_t CoutMutex;    //блокування виводу

void* Philosopher(void* data)  //функція потоку філософа
{
    int count = 0;   //лічильник кількості обідів філософа
    int philosopher_number = *(int*)data;   //номер філософа
```



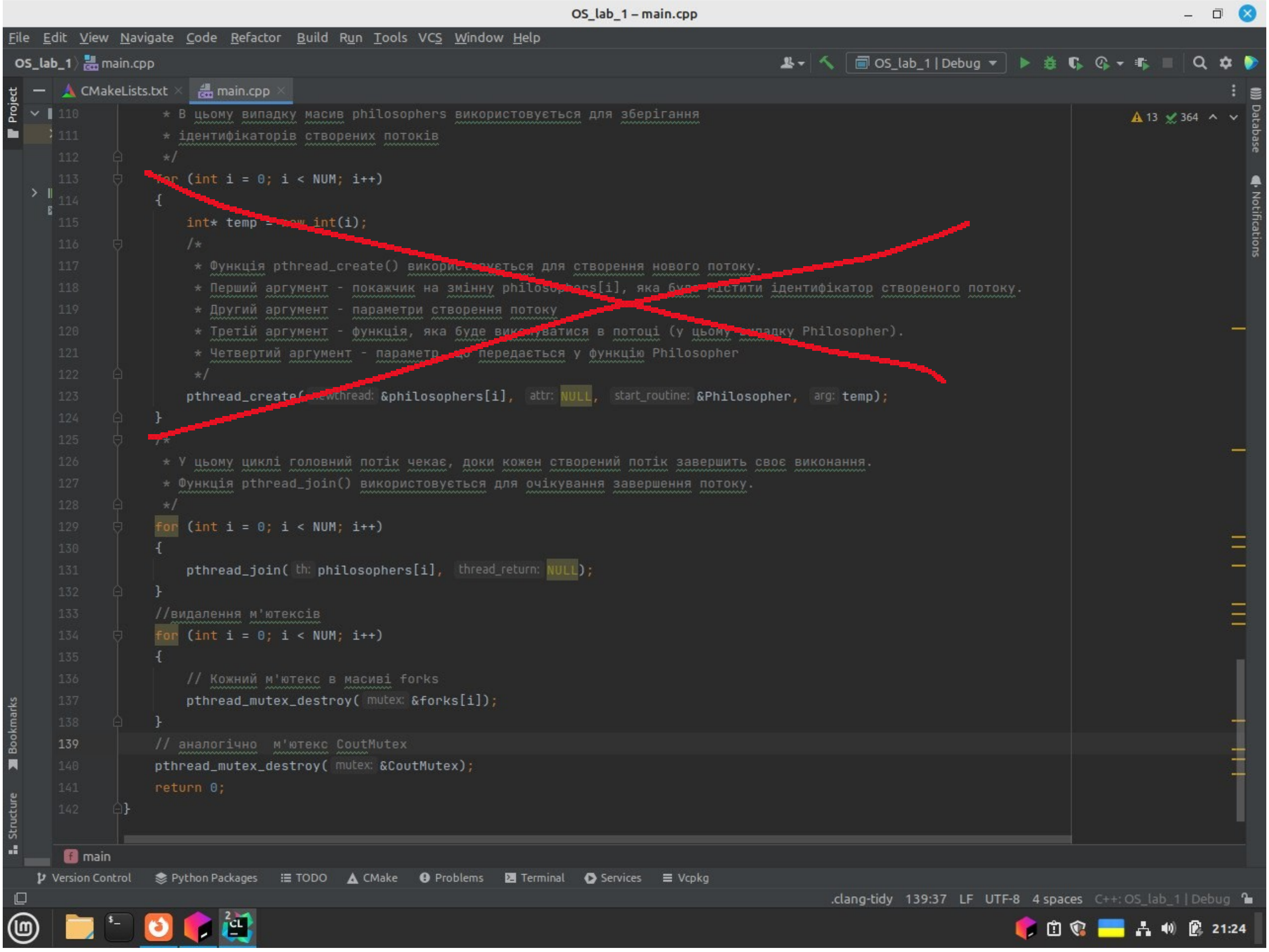
```
void* Philosopher(void* data)  //функція потоку філософа
{
    int count = 0;   //лічильник кількості обідів філософа
    int philosopher_number = *(int*)data;   //номер філософа
    /*
     * ight_fork - індекс правої виделки для даного філософа (обчислюється за допомогою оператора %),
     * left_fork - індекс лівої виделки для даного філософа.
     */
    int right_fork = (philosopher_number + 1) % NUM; //права виделка
    int left_fork = (philosopher_number) % NUM;   //ліва виделка

    sleep( seconds: rand() % 5); /*випадкове затримку перед початком обіду філософа*/

    while (count < MEALS_NUM) //філософ їсть вказану кількість разів
    {
        /*чи може поточний філософ заблокувати ліву виделку. pthread_mutex_trylock()
         * намагається заблокувати м'ютекс, але не блокується,
         * якщо м'ютекс вже заблокований іншим потоком. Якщо trylock успішний (м'ютекс вільний),
         * філософ бере ліву виделк*/
       if (!pthread_mutex_trylock( mutex: &forks[left_fork]))  //бере ліву виделку
        {
            /*
             * перевіряє, чи може поточний філософ заблокувати праву виделку.
             * Якщо trylock успішний (м'ютекс вільний), філософ бере праву виделку.
             */
            if (!pthread_mutex_trylock( mutex: &forks[right_fork]))    //бере праву виделку
             {
                /*
                 * блокують м'ютекс CoutMutex, щоб заблокувати доступ інших потоків до виводу в консоль.
                 * Потім виводиться повідомлення про початок обіду філософа, яке включає його номер.
                 * Після виводу м'ютекс CoutMutex розблоковується, дозволяючи іншим потокам
                 * отримати доступ до виводу в консоль.
```



```
                * блокують м'ютекс CoutMutex, щоб заблокувати доступ інших потоків до виводу в консоль.
                * Потім виводиться повідомлення про початок обіду філософа, яке включає його номер.
                * Після виводу м'ютекс CoutMutex розблоковується, дозволяючи іншим потокам
                * отримати доступ до виводу в консоль.
                */
                pthread_mutex_lock( mutex: &CoutMutex); //блокування виводу в консоль
                cout << "Philosopher " << philosopher_number + 1 << " start eating" << endl;
                pthread_mutex_unlock( mutex: &CoutMutex);         //розблокування виводу в консоль
                sleep( seconds: rand() % 5);   //час, що філософ їсть

                /*
                 * блокують м'ютекс CoutMutex, виводять повідомлення
                 * про закінчення обіду філософа, включаючи
                 * кількість прийомів їжі, і розблоковують м'ютекс CoutMutex
                 */
                pthread_mutex_lock( mutex: &CoutMutex);
                count++;
                cout << "Philosopher " << philosopher_number + 1 << " stop eating (" << count << "/" << MEALS_NUM << ")" << endl;
                pthread_mutex_unlock( mutex: &CoutMutex);
                /*
                 * Ці рядки розблоковують виделки, відпускаючи їх для інших філософів.
                 */
                pthread_mutex_unlock( mutex: &forks[right_fork]);  //відпускає праву виделку
                }
            pthread_mutex_unlock( mutex: &forks[left_fork]);  //відпускає ліву виделку
            }
        sleep( seconds: rand() % 10);
        }
}

int main()
{
    srand( seed: time( timer: NULL)); // для генерування випадкових чисел

    // далі в нас йде ініціалізація м'ютексів
```



```
int main()
{
    srand( seed: time( timer: NULL)); // для генерування випадкових чисел

    // далі в нас йде ініціалізація м'ютексів
    /*
     *  Кожна виделка (fork) має свій власний м'ютекс, представлений у масиві forks.
     *  Функція pthread_mutex_init() використовується для ініціалізації м'ютекса.
     *  Перший аргумент - покажчик на м'ютекс, а другий аргумент - параметри ініціалізації
     */
    for (int i = 0; i < NUM; i++)
    {
        pthread_mutex_init( mutex: &forks[i],  mutexattr: NULL);
    }
    pthread_mutex_init( mutex: &CoutMutex,  mutexattr: NULL);

    //ініціалізація потоків
    pthread_t philosophers[NUM];
    /*
     * У цьому циклі створюються потоки філософів.
     * Кожен потік представлений змінною типу pthread_t.
     * В цьому випадку масив philosophers використовується для зберігання
     * ідентифікаторів створених потоків
     */
    for (int i = 0; i < NUM; i++)
    {
        int* temp = new int(i);
        /*
         * Функція pthread_create() використовується для створення нового потоку.
         * Перший аргумент - покажчик на змінну philosophers[i], яка буде містити ідентифікатор створеного потоку.
         * Другий аргумент - параметри створення потоку
         * Третій аргумент - функція, яка буде виконуватися в потоці (у цьому випадку Philosopher).
         * Четвертий аргумент - параметр, що передається у функцію Philosopher
         */
```



```
     * В цьому випадку масив philosophers використовується для зберігання
     * ідентифікаторів створених потоків
     */
    for (int i = 0; i < NUM; i++)
    {
        int* temp = new int(i);
        /*
         * Функція pthread_create() використовується для створення нового потоку.
         * Перший аргумент - покажчик на змінну philosophers[i], яка буде містити ідентифікатор створеного потоку.
         * Другий аргумент - параметри створення потоку
         * Третій аргумент - функція, яка буде використовуватися в потоці (у цьому випадку Philosopher).
         * Четвертий аргумент - параметр, що передається у функцію Philosopher
         */
        pthread_create( newthread: &philosophers[i],  attr: NULL,  start_routine: &Philosopher,  arg: temp);
    }
    /*
     * У цьому циклі головний потік чекає, доки кожен створений потік завершить своє виконання.
     * Функція pthread_join() використовується для очікування завершення потоку.
     */
    for (int i = 0; i < NUM; i++)
    {
        pthread_join( th: philosophers[i],  thread_return: NULL);
    }
    //видалення м'ютексів
    for (int i = 0; i < NUM; i++)
    {
        // Кожний м'ютекс в масиві forks
        pthread_mutex_destroy( mutex: &forks[i]);
    }
    // аналогічно  м'ютекс CoutMutex
    pthread_mutex_destroy( mutex: &CoutMutex);
    return 0;
}
```